AMRITA VISHWA VIDYAPEETHAM

Department of Artificial Intelligence & Computer Science | School of Engineering

Academic Course: Database Management Systems (19AIE301) • Semester 5

PlaceX: College Placement Management Portal

A Centralized, BCNF-Normalized Relational Database & Full-Stack Portal for Campus Recruitment Management

Project Identification	**Team Number 7** • DBMS Course Project
Academic Year & Term	Fifth Semester (B.Tech AIE / CSE), 2025–2026
Database Engine	PostgreSQL Relational DBMS (v14+)
Application Stack	Node.js • Express.js • Vanilla JavaScript • Modern Glassmorphism CSS3
Primary GitHub Repository	`https://github.com/K-LAXMIVIGNESH/Collage-Placement-Portal`

2. TEAM MEMBERS

This project was conceptualized, designed, implemented, and benchmarked by Team Number 7. Individual roles and primary responsibilities are detailed below:

S.No	Student Name	Roll Number	Core Contributions & Responsibilities
1	**Rupa Kandula**	`CB.SC.U4AIE24122`	Conceptual Database Modeling, Entity-Relationship (ER) Architecture, Formal UML Class Diagram Design, and BCNF Normalization Dependency Proofs.
2	**Laxmi Vignesh**	`CB.SC.U4AIE24124`	Physical Relational Schema Implementation in PostgreSQL, Referential Integrity & Cascade Constraint Formulation, SQL Query Optimization, and Node.js/Express API Engine.
3	**Varalaxmi K S D**	`CB.SC.U4AIE24125`	Frontend Dashboard Architecture, Dynamic Search and Filtering Module Development, Client-Side API Integration, Data Validation, and Quality Assurance.

Project Collaboration & Repository Management

All artifacts, database schemas, API controllers, and frontend single-page application modules are version-controlled and hosted on the team's official GitHub repository: **K-LAXMIVIGNESH/Collage-Placement-Portal**.

3. INTRODUCTION AND PROBLEM STATEMENT

3.1 Introduction and Background

In contemporary universities and higher education institutions, campus placement is a vital metric of institutional excellence and student career development. Coordinating placement drives entails orchestrating high-velocity interactions between thousands of student aspirants, hundreds of multinational corporate recruiters, and the university placement directorate.

Recruitment seasons involve publishing diverse job descriptions, establishing multifaceted eligibility criteria (ranging from minimum CGPA thresholds and allowable active backlog limits to branch restrictions), processing thousands of drive-specific applications, coordinating sequential interview evaluation rounds (online coding assessments, technical interviews, managerial rounds, HR evaluations), and managing final offer distributions.

PlaceX is an enterprise-grade, centralized College Placement Management Portal developed to replace antiquated placement workflows. Built on a robust, normalized PostgreSQL relational database and connected to an asynchronous Node.js/Express REST backend with a modern glassmorphism web user interface, PlaceX consolidates student academic profiles, corporate recruitment campaigns, multi-stage application lifecycles, and institutional analytics into one unified ecosystem.

3.2 Problem Statement

Despite the criticality of campus placements, conventional institutional administration continues to rely heavily on fragmented spreadsheets (Microsoft Excel, Google Sheets), manual email exchanges, and physical bulletin boards. This legacy approach suffers from several severe limitations:

1. **Data Redundancy & Relational Anomalies:** Student records and academic scores are repeatedly duplicated across disparate department sheets. This introduces severe insertion, update, and deletion anomalies. For instance, updating a student's active backlog status or phone number in one department sheet fails to propagate to the central placement repository, leading to corrupted institutional data.
2. **Error-Prone, Manual Eligibility Verification:** Every visiting company enforces distinct eligibility standards based on minimum CGPA, maximum backlogs, and allowable disciplines. Placement cell coordinators are forced to manually cross-reference thousands of candidate records against each company cutoff. This manual parsing is labor-intensive, slow, and prone to human error—resulting in eligible candidates being inadvertently disqualified or ineligible candidates attending recruitment drives.
3. **Absence of Multi-Stage Application Lifecycle Tracking:** Traditional spreadsheets offer zero real-time visibility into candidate progression through sequential interview stages (Applied → Eligible → Shortlisted → Technical Round → Managerial Round → HR Round → Offer Extended → Offer Accepted). Both students and coordinators remain unaware of candidate statuses in the evaluation pipeline.
4. **Lack of Real-Time Placement Intelligence & Analytics:** College administrators cannot instantly query institutional placement KPIs—such as overall placement percentages, highest and average CTCs, branch-wise recruitment distributions, and recruiter offer acceptance rates. Compiling reports requires days of manual data aggregation.

3.3 Proposed Solution & System Objectives

To resolve these operational bottlenecks, **PlaceX** introduces an integrated relational system designed to satisfy the following primary engineering objectives:

- **Centralized Relational Architecture:** Design and populate a unified PostgreSQL database seeded with over 12,000 realistic student academic records and comprehensive enterprise profiles.

- **Guaranteed Relational Integrity & BCNF Compliance:** Eliminate data redundancy by decomposing multi-valued attributes and ensuring every relation satisfies Boyce-Codd Normal Form (BCNF) with strict primary, composite, and foreign key constraints.
- **Automated Dynamic Eligibility Validation:** Implement an eligibility engine that dynamically compares student CGPA, backlogs, and academic branches against drive cutoffs without human intervention.
- **End-to-End Application & Interview Pipeline Tracking:** Maintain real-time auditability across all recruitment phases from initial drive publication to offer letter acceptance.
- **High-Performance Analytical Querying:** Provide optimized SQL query templates and dashboard APIs to compute institutional metrics, compensation averages, and recruiter drive summaries.
- **Role-Based Web Dashboards:** Provide custom portals for Students (profile, eligible drives, application status, offer management) and Placement Officers (KPI dashboard, candidate search, company directory, and drive tracking).

4. CONCEPTUAL AND LOGICAL DATABASE DESIGN

4.1 Database Design Methodology

The database architecture of PlaceX was developed following a disciplined 4-stage engineering lifecycle:

1. **Requirements Analysis:** Formalized business rules, entity attributes, cardinality constraints, and administrative reporting requirements.
2. **Conceptual Modeling:** Mapped real-world entities into an Entity-Relationship (ER) model and formal Unified Modeling Language (UML) Class Diagram.
3. **Logical Design & Normalization:** Translated conceptual entities into 11 relational schemas, eliminated multi-valued dependencies, and proved BCNF compliance.
4. **Physical Implementation (DDL):** Engineered PostgreSQL DDL scripts specifying exact domain types, `NOT NULL` constraints, `CHECK` conditions, unique constraints, and `ON DELETE CASCADE` rules.

4.2 Conceptual Modeling (Entities, Relationships & Cardinalities)

The conceptual schema models the placement ecosystem across 11 distinct entities:

- `student`: Captures student identity, academic credentials (CGPA, backlogs, branch, degree), technical proficiencies (coding, communication, aptitude, certifications), and placement outcome.
- `company`: Stores participating enterprise details, corporate sector, headquarters location, website, and talent acquisition contacts.
- `placement_officer`: Represents institutional administrators responsible for reviewing applications, scheduling interviews, and managing drives.
- `placement_drive`: Represents an individual campus recruitment campaign conducted by an enterprise for a specified job role, CTC package, and date.
- `eligibility`: Specifies drive-specific cutoffs including minimum CGPA and maximum allowable backlogs (enforces a strict 1:1 relationship with `placement_drive` via a `UNIQUE` constraint).
- `eligibility_allowed_branch`: Resolves the multi-valued allowed branch attribute, mapping allowable disciplines to eligibility criteria via a composite key.
- `hr`: Captures recruiter personnel associated with participating companies.
- `applies`: Models the many-to-many (M:N) associative relationship between students and placement drives.
- `application`: Captures individual formal application submissions, tracking progression dates, assigned placement officers, and status updates.
- `interview`: Represents sequential interview rounds (Technical, Managerial, HR) conducted under an application, storing dates, outcomes, and committee remarks.
- `offer`: Records final placement compensation packages, offer issuance dates, and student decision statuses (Accepted, Rejected, Pending).

Cardinality Constraints Summary

Entity Relationship	Cardinality	Business Rule & Integrity Constraint
`company` → `placement_drive`	1 : M (One to Many)	A company conducts one or more recruitment drives; each drive belongs to exactly one company.

Entity Relationship	Cardinality	Business Rule & Integrity Constraint
`placement_drive` → `eligibility`	1 : 1 (One to One)	Every placement drive specifies exactly one eligibility rule profile (enforced via `drive_id UNIQUE`).
`eligibility` → `eligibility_allowed_branch`	1 : M (One to Many)	One eligibility rule can permit multiple academic engineering branches (CSE, IT, ECE, EEE).
`student` ↔ `placement_drive` (`applies`)	M : N (Many to Many)	A student may apply to multiple drives; each drive accumulates applications from multiple candidates.
`student` → `application`	1 : M (One to Many)	A student submits multiple drive-specific applications.
`application` → `interview`	1 : M (One to Many)	An application progresses through sequential evaluation rounds (Technical, Managerial, HR).
`student` → `offer`	1 : M (One to Many)	A student may receive multiple offers from distinct recruiting organizations.
`company` → `offer`	1 : M (One to Many)	A corporate recruiter extends job offers to successful candidates.

5. NORMALIZATION AND FUNCTIONAL DEPENDENCIES

5.1 Objective of Relational Normalization

Normalization is the formal mathematical process of decomposing complex relations into well-structured schemas to minimize data redundancy, eradicate insertion, update, and deletion anomalies, and guarantee both **lossless join decomposition** and **functional dependency preservation**.

5.2 Evolution from Review 1 to Final Schema (1NF Violation Analysis)

During the preliminary design phase (Review 1), the candidate eligibility schema was formulated as:

```
eligibility(eligibility_id, drive_id, minimum_cgpa, maximum_backlogs, allowed_branch)
```

where the `allowed_branch` attribute contained non-atomic, comma-delimited strings such as `'CSE, IT, ECE'`.

- **First Normal Form (1NF) Violation:** 1NF requires that the domain of every attribute must contain only atomic (indivisible) values, and each tuple must hold exactly one value from that domain. Storing comma-separated lists directly violates atomicity and forces the DBMS to rely on slow, inefficient substring pattern matching (`LIKE '%CSE%'`) instead of indexed relational joins.
- **Decomposition to Achieve 1NF & BCNF:** The multi-valued dependency was resolved by decomposing the relation into two distinct schemas:
  1. `eligibility(eligibility_id, drive_id, minimum_cgpa, maximum_backlogs)`
  2. `eligibility_allowed_branch(eligibility_id, allowed_branch)` with composite primary key `(eligibility_id, allowed_branch)` and foreign key referencing `eligibility(eligibility_id) ON DELETE CASCADE`.

5.3 Step-by-Step Justification of Normal Forms

- **First Normal Form (1NF):** All attributes across all 11 relations contain exclusively atomic values. No repeating groups or delimited strings exist. Every relation possesses a uniquely identifiable primary key.
- **Second Normal Form (2NF):** The schema is in 1NF and contains no *partial functional dependencies*. In composite-key relations (`applies(student_id, drive_id)` and `eligibility_allowed_branch(eligibility_id, allowed_branch)`), every non-prime attribute (where applicable) is fully functionally dependent on the entire composite key.
- **Third Normal Form (3NF):** The schema is in 2NF and exhibits no *transitive functional dependencies* (where $X ightarrow Y$ and $Y ightarrow Z$, with Z depending on X through non-prime attribute Y). All non-prime attributes depend directly and solely on candidate keys.
- **Boyce-Codd Normal Form (BCNF):** A relation R is in BCNF if and only if for every non-trivial functional dependency $X ightarrow Y$, the determinant X is a **superkey**. Under the verified candidate keys, every determinant in the PlaceX schema is a candidate key.

5.4 Functional Dependency & Candidate Key Analysis

Relation Schema	Candidate Key(s)	Verified Functional Dependencies (FDs)	Normal Form
student	`student_id`	`student_id → (name, gender, age, degree, branch, cgpa, backlogs, skills, placement_status, package_lpa, email, phone, password, city, state)`	**BCNF**
company	`company_id`	`company_id → (company_name, company_type, industry, location, website, email, phone)`	**BCNF**
placement_officer	`officer_id`	`officer_id → (officer_name, email, phone, password)`	**BCNF**
placement_drive	`drive_id`	`drive_id → (company_id, job_role, package_lpa, vacancies, drive_date, last_date_to_apply)`	**BCNF**
eligibility	`eligibility_id`, `drive_id`	`eligibility_id → (drive_id, minimum_cgpa, maximum_backlogs)` `drive_id → (eligibility_id, minimum_cgpa, maximum_backlogs)` (via `UNIQUE`)	**BCNF**
eligibility_allowed_branch	`(eligibility_id, allowed_branch)`	All attributes are prime; composite key uniquely identifies permitted disciplines.	**BCNF**
hr	`hr_id`	`hr_id → (company_id, hr_name, email, phone)`	**BCNF**
applies	`(student_id, drive_id)`	Composite key uniquely identifies candidate-drive application pairing.	**BCNF**
application	`application_id`	`application_id → (student_id, drive_id, officer_id, application_date, application_status)`	**BCNF**
interview	`interview_id`	`interview_id → (application_id, interview_round, interview_date, result, remarks)`	**BCNF**
offer	`offer_id`	`offer_id → (student_id, company_id, offer_date, package_lpa, offer_status)`	**BCNF**

6. ADVANCED SQL IMPLEMENTATION AND QUERY DESIGN

The PlaceX system relies on advanced SQL operations executing multi-table joins, conditional aggregations, subqueries, grouping with `HAVING` filters, and referential cascade modifications. The 10 core production queries are detailed below:

6.1 Dynamic Multi-Table Eligibility Verification Query (5-Table Join)

DBMS Concept: Complex Multi-Table Inner Join with Conjunction Filtering across 5 normalized relations.

```
SELECT
    s.student_id,
    s.name AS student_name,
    s.branch,
    s.cgpa,
    s.backlogs,
    c.company_name,
    pd.job_role,
    pd.package_lpa,
    e.minimum_cgpa,
    e.maximum_backlogs
FROM student s
JOIN eligibility_allowed_branch eab
    ON s.branch = eab.allowed_branch
JOIN eligibility e
    ON eab.eligibility_id = e.eligibility_id
JOIN placement_drive pd
    ON e.drive_id = pd.drive_id
JOIN company c
    ON pd.company_id = c.company_id
WHERE s.cgpa >= e.minimum_cgpa
  AND s.backlogs <= e.maximum_backlogs
ORDER BY pd.package_lpa DESC, s.cgpa DESC;
```

6.2 End-to-End Application Lifecycle Tracking (4-Table Join)

DBMS Concept: Multi-Table Relational Projection across Student, Drive, Company, and Administrative Coordinator.

```
SELECT
    a.application_id,
    s.name AS student_name,
    s.branch,
    c.company_name,
    pd.job_role,
    po.officer_name AS assigned_coordinator,
    a.application_date,
    a.application_status
FROM application a
JOIN student s ON a.student_id = s.student_id
JOIN placement_drive pd ON a.drive_id = pd.drive_id
JOIN company c ON pd.company_id = c.company_id
JOIN placement_officer po ON a.officer_id = po.officer_id
ORDER BY a.application_date DESC;
```

6.3 Multi-Stage Interview Pipeline Monitoring (5-Table Join)

DBMS Concept: Relational Join linking Sequential Interview Rounds to Application and Recruiter Entities.

```
SELECT
    i.interview_id,
    s.name AS candidate_name,
    c.company_name,
    pd.job_role,
    i.interview_round,
    i.interview_date,
    i.result,
    i.remarks
FROM interview i
JOIN application a ON i.application_id = a.application_id
JOIN student s ON a.student_id = s.student_id
JOIN placement_drive pd ON a.drive_id = pd.drive_id
JOIN company c ON pd.company_id = c.company_id
ORDER BY i.interview_date DESC;
```

6.4 Placement Performance & CTC Summary Analytics (Conditional Aggregations)

DBMS Concept: Aggregate Functions (`COUNT` , `AVG` , `MAX`) with Inline Conditional `CASE` Statements.

```
SELECT
    COUNT(student_id) AS total_students,
    COUNT(CASE WHEN LOWER(placement_status) = 'placed' THEN 1 END) AS placed_students,
    ROUND(
        (COUNT(CASE WHEN LOWER(placement_status) = 'placed' THEN 1 END)::numeric /
        COUNT(student_id)::numeric) * 100,
        2
    ) AS placement_percentage,
    ROUND(AVG(CASE WHEN package_lpa > 0 THEN package_lpa END), 2) AS average_package_lpa,
    MAX(package_lpa) AS highest_package_lpa
FROM student;
```

6.5 Branch-Wise Placement & Package Distribution (GROUP BY & HAVING)

DBMS Concept: Relational Grouping with Post-Aggregation Filtering via `HAVING` Clause.

```
SELECT
    branch,
    COUNT(student_id) AS total_candidates,
    COUNT(CASE WHEN LOWER(placement_status) = 'placed' THEN 1 END) AS placed_candidates,
    ROUND(AVG(CASE WHEN package_lpa > 0 THEN package_lpa END), 2) AS avg_branch_package,
    MAX(package_lpa) AS max_branch_package
FROM student
GROUP BY branch
HAVING COUNT(student_id) > 10
ORDER BY avg_branch_package DESC NULLS LAST;
```

6.6 Placement Drive Compensation Evaluation (SUM, MAX, MIN)

DBMS Concept: Statistical Aggregation on Drive Remuneration Attributes.

```
SELECT
    SUM(package_lpa) AS total_package_lpa,
    MAX(package_lpa) AS highest_package,
    MIN(package_lpa) AS lowest_package
FROM placement_drive;
```

6.7 Company-Wise Average Package Offered (JOIN, GROUP BY)

DBMS Concept: Multi-Table Aggregation Grouped by Enterprise Recruiter.

```
SELECT
    c.company_name,
    ROUND(AVG(pd.package_lpa), 2) AS average_package
FROM company c
JOIN placement_drive pd ON c.company_id = pd.company_id
GROUP BY c.company_id, c.company_name
ORDER BY average_package DESC;
```

6.8 Application Status Distribution Breakdown (COUNT with GROUP BY)

DBMS Concept: Categorical Distribution Analysis on Candidate Progression Stages.

```
SELECT
    application_status,
    COUNT(*) AS total_applications
FROM application
GROUP BY application_status
ORDER BY total_applications DESC;
```

6.9 Multi-Drive Recruiter Identification (GROUP BY & HAVING > 1)

DBMS Concept: Identifying Enterprise Partners Conducting Multiple Recruitment Drives.

```
SELECT
    c.company_name,
    COUNT(pd.drive_id) AS total_drives
FROM company c
JOIN placement_drive pd ON c.company_id = pd.company_id
GROUP BY c.company_id, c.company_name
HAVING COUNT(pd.drive_id) > 1;
```

6.10 Referential Cascade Deletion Trigger Implementation (DDL Constraint)

DBMS Concept: Altering Table Constraints to Enforce Referential `ON DELETE CASCADE` Integrity.

```
ALTER TABLE eligibility_allowed_branch
DROP CONSTRAINT eligibility_allowed_branch_eligibility_id_fkey;

ALTER TABLE eligibility_allowed_branch
ADD CONSTRAINT eligibility_allowed_branch_eligibility_id_fkey
FOREIGN KEY (eligibility_id)
REFERENCES eligibility(eligibility_id)
ON DELETE CASCADE;
```

7. ER DIAGRAM AND UML DIAGRAM

7.1 Conceptual Entity-Relationship (ER) Diagram

The conceptual ER diagram models the entities, multi-valued decompositions, associative relationships, and key constraints designed during the formal modeling phase.

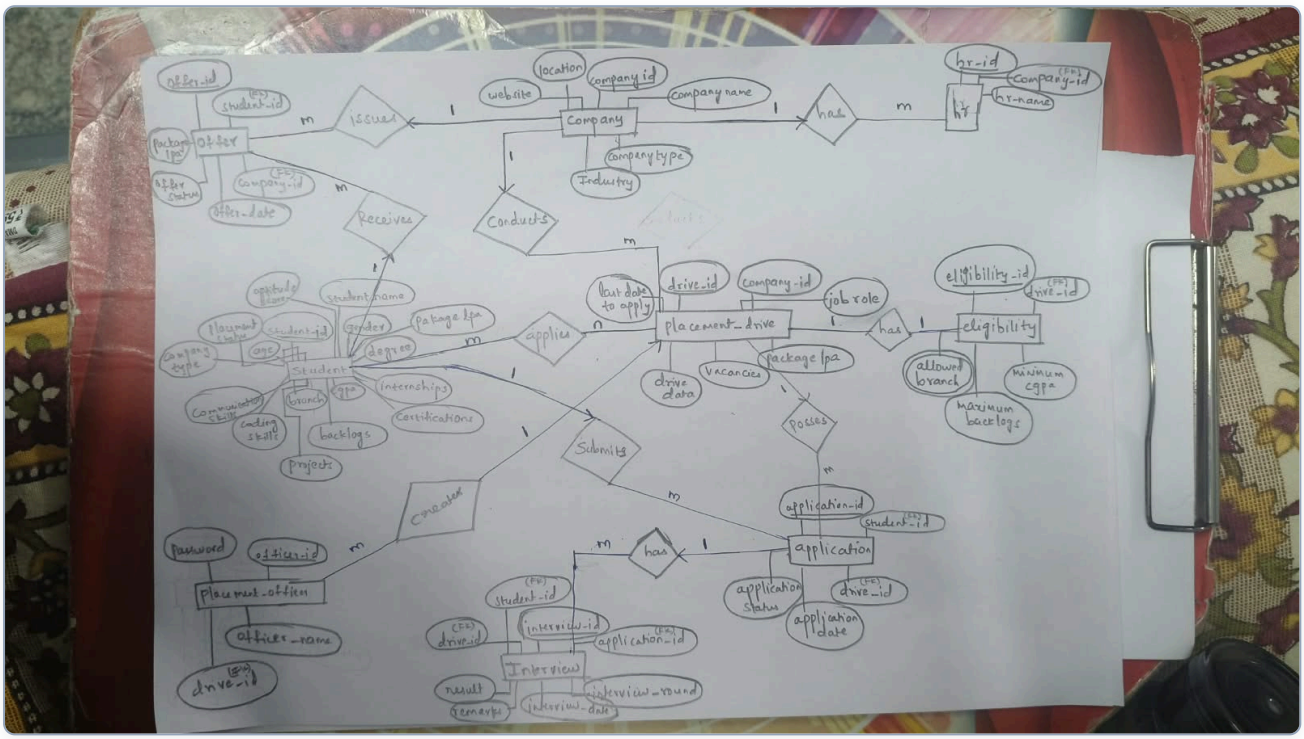


Figure 7.1: Conceptual Entity-Relationship (ER) Diagram illustrating Entities, Relationships, and Cardinality Constraints.

> **Key Architectural Highlights of the ER Diagram**
>
> - **Associative Entity:** `applies` resolves the M:N cardinality between `student` and `placement_drive`.
> - **1:1 Enforced via Unique Constraint:** `placement_drive` relates 1:1 to `eligibility` by designating `eligibility.drive_id` as `UNIQUE`.
> - **Decomposed Multi-Valued Attribute:** Allowed disciplines are modeled via `eligibility_allowed_branch` with composite primary key `(eligibility_id, allowed_branch)`.

7.2 Formal UML Class Diagram

The UML Class Diagram specifies object-oriented class representations, attribute typings, visibility indicators (`+` public), and structural associations.

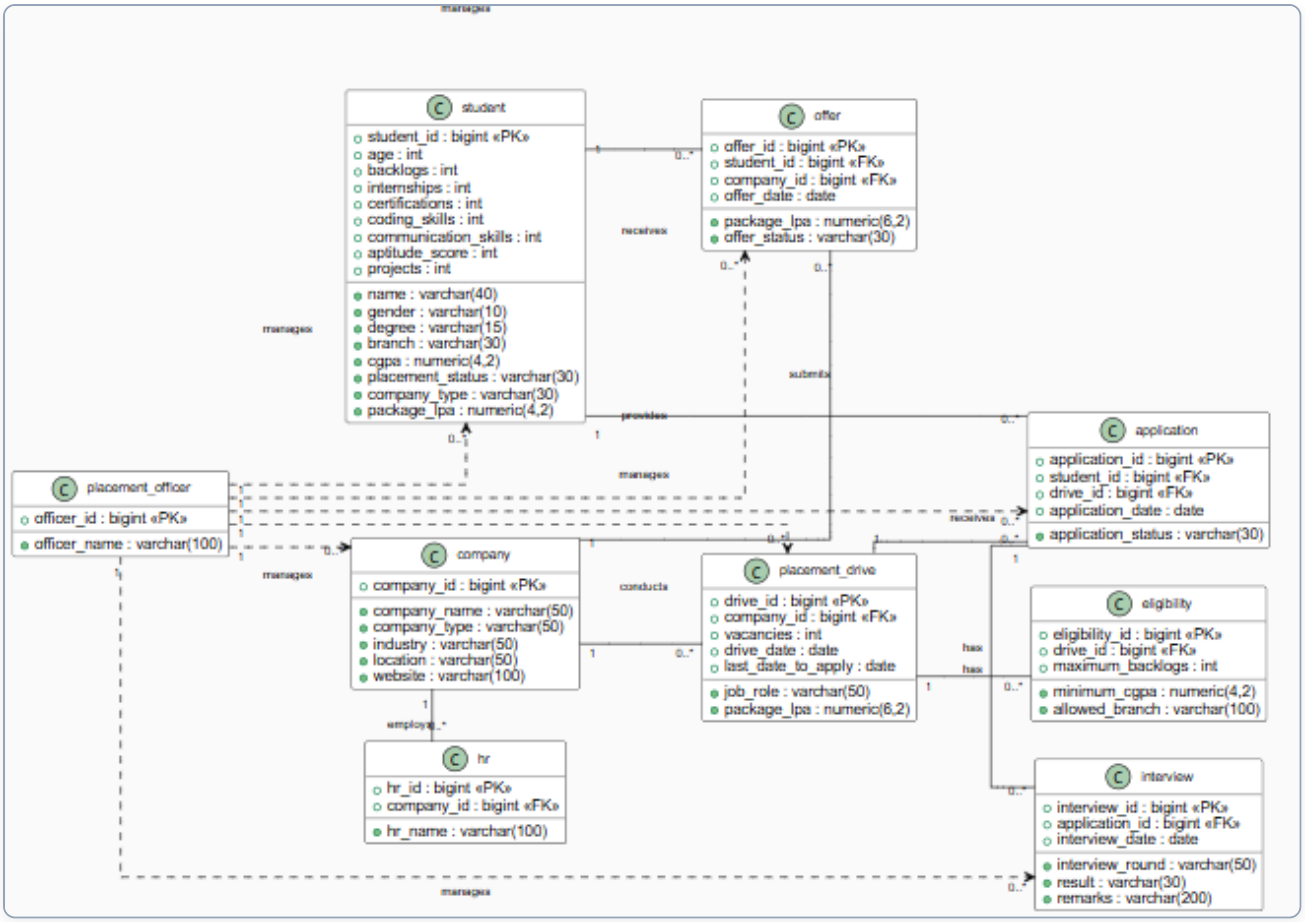


Figure 7.2: Formal Unified Modeling Language (UML) Class Diagram of the PlaceX Ecosystem.

8. RELATIONAL SCHEMA

The conceptual model maps to 11 normalized relational schemas. Primary keys are underlined (**PK**), composite primary keys are explicitly identified, and foreign keys point to their referenced parent relations:

1. Student Relation

```
Student(student_id, name, gender, age, degree, branch, cgpa, backlogs, internships, certifications, coding_skills, communication_skills, aptitude_score, projects, placement_status, company_type, package_lpa, email, phone, password, city, state)
```

2. Company Relation

```
Company(company_id, company_name, company_type, industry, location, website, email, phone)
```

3. Placement_Officer Relation

```
Placement_Officer(officer_id, officer_name, email, phone, password)
```

4. Placement_Drive Relation

```
Placement_Drive(drive_id, company_id*, job_role, package_lpa, vacancies, drive_date, last_date_to_apply)
  • Foreign Key: company_id → Company(company_id)
```

5. Eligibility Relation

```
Eligibility(eligibility_id, drive_id*, minimum_cgpa, maximum_backlogs)
  • Foreign Key: drive_id → Placement_Drive(drive_id) [UNIQUE]
```

6. Eligibility_Allowed_Branch Relation

```
Eligibility_Allowed_Branch(eligibility_id*, allowed_branch)
  • Composite Primary Key: (eligibility_id, allowed_branch)
  • Foreign Key: eligibility_id → Eligibility(eligibility_id) ON DELETE CASCADE
```

7. HR Relation

```
HR(hr_id, company_id*, hr_name, email, phone)
  • Foreign Key: company_id → Company(company_id)
```

8. Applies Relation (Associative Mapping)

```
Applies(student_id*, drive_id*)
  • Composite Primary Key: (student_id, drive_id)
  • Foreign Keys: student_id → Student(student_id), drive_id → Placement_Drive(drive_id)
```

9. Application Relation

```
Application(application_id, student_id*, drive_id*, officer_id*, application_date, application_status)
  • Foreign Keys: student_id → Student(student_id), drive_id → Placement_Drive(drive_id), officer_id → Placement_Officer(officer_id)
```

10. Interview Relation

```
Interview(interview_id, application_id*, interview_round, interview_date, result, remarks)
  • Foreign Key: application_id → Application(application_id)
```

11. Offer Relation

```
Offer(offer_id, student_id*, company_id*, offer_date, package_lpa, offer_status)
  • Foreign Keys: student_id → Student(student_id), company_id → Company(company_id)
```

9. DATABASE TABLES

This section specifies the exact column definitions, domain types, constraints, and sample tuples for each of the 11 database relations.

9.1 Table: student (Total Seed: 12,000+ Records)

Column Name	Data Type	Constraints	Description
student_id	BIGINT	PRIMARY KEY	Unique institutional student identification number
name	VARCHAR(40)	NOT NULL	Full name of the student
gender	VARCHAR(10)	NOT NULL, CHECK (gender IN ('Male','Female'))	Student gender domain restriction
age	INTEGER	NOT NULL	Student age in years
degree	VARCHAR(15)	NOT NULL	Enrolled academic degree (e.g., B.Tech)
branch	VARCHAR(30)	NOT NULL	Major academic engineering branch (CSE, IT, ECE, EEE)
cgpa	NUMERIC(4,2)	NOT NULL	Cumulative Grade Point Average (0.00 – 10.00)
backlogs	INTEGER	NOT NULL	Count of currently active academic backlogs
internships	INTEGER	NOT NULL	Number of completed industrial internships
certifications	INTEGER	NOT NULL	Count of verified technical certifications
coding_skills	INTEGER	NOT NULL	Standardized coding proficiency score (1 – 10)
communication_skills	INTEGER	NOT NULL	Communication assessment score (1 – 10)
aptitude_score	INTEGER	NOT NULL	Quantitative & logical aptitude score (1 – 100)
projects	INTEGER	NOT NULL	Number of documented academic/industrial projects
placement_status	VARCHAR(30)	NOT NULL	Recruitment status: 'Placed', 'Not Placed'
company_type	VARCHAR(30)	—	Hiring corporate classification ('IT', 'Consulting')
package_lpa	NUMERIC(6,2)	—	Secured compensation in Lakhs Per Annum (LPA)
email	VARCHAR(100)	—	University student email address
phone	VARCHAR(15)	—	Contact telephone number
password	VARCHAR(100)	—	Portal authentication credential hash
city, state	VARCHAR(50)	—	Residential location details

Sample Tuples (student)

ID	Name	Branch	CGPA	Backlogs	Coding	Status	CTC (LPA)	City
1	Aarav Sharma	CSE	8.85	0	9	Placed	12.00	Hyderabad

ID	Name	Branch	CGPA	Backlogs	Coding	Status	CTC (LPA)	City
2	Ananya Iyer	IT	7.40	0	7	Placed	7.50	Bangalore
3	Rohan Verma	ECE	6.75	1	6	Not Placed	0.00	Chennai
4	Meera Nair	CSE	8.20	0	8	Placed	8.00	Kochi
5	Vikram Reddy	EEE	9.10	0	10	Placed	15.00	Hyderabad

9.2 Table: company

Column Name	Data Type	Constraints	Description
company_id	BIGINT	PRIMARY KEY	Unique corporate identification number
company_name	VARCHAR(50)	NOT NULL	Registered company name
company_type	VARCHAR(50)	NOT NULL	Sector classification ('IT', 'Consulting')
industry	VARCHAR(50)	—	Industry domain (e.g. 'Information Technology')
location	VARCHAR(50)	—	Corporate office location
website	VARCHAR(100)	—	Official corporate portal URL
email, phone	VARCHAR(100/15)	—	Talent acquisition contact channels

Sample Tuples (company)

ID	Company Name	Sector	Industry	Location	Website	Email
101	TCS	IT	Information Technology	Hyderabad	www.tcs.com	hr@tcs.com
102	Infosys	IT	Information Technology	Bangalore	www.infosys.com	hr@infosys.com
103	Deloitte	Consulting	Consulting	Hyderabad	www.deloitte.com	hr@deloitte.com
104	Amazon	IT	E-Commerce	Bangalore	www.amazon.com	hr@amazon.com
105	Wipro	IT	Information Technology	Chennai	www.wipro.com	hr@wipro.com

9.3 Table: placement_officer

Column Name	Data Type	Constraints	Description
officer_id	BIGINT	PRIMARY KEY	Unique placement administrator ID
officer_name	VARCHAR(100)	NOT NULL	Full name of placement officer
email	VARCHAR(100)	NOT NULL	Institutional email address
phone	VARCHAR(15)	—	Mobile contact number
password	VARCHAR(100)	NOT NULL	Portal access authentication hash

Sample Tuples (placement_officer)

Officer ID	Officer Name	Email	Phone
801	Anita Reddy	anita@college.com	9876600001
802	Ravi Kumar	ravi@college.com	9876600002
803	Vignesh	laxmivigneshkonkimalla@gmail.com	9490517699

9.4 Table: placement_drive

Column Name	Data Type	Constraints	Description
drive_id	BIGINT	PRIMARY KEY	Unique campus placement drive ID
company_id	BIGINT	FOREIGN KEY (company)	Recruiting enterprise conducting drive
job_role	VARCHAR(50)	NOT NULL	Designation offered to candidates
package_lpa	NUMERIC(6,2)	NOT NULL	Cost to Company (CTC) offered in LPA
vacancies	INTEGER	NOT NULL	Number of open hiring positions
drive_date	DATE	NOT NULL	Scheduled date of placement drive
last_date_to_apply	DATE	NOT NULL	Registration deadline for students

Sample Tuples (placement_drive)

Drive ID	Company	Job Role	CTC (LPA)	Vacancies	Drive Date	Deadline
1001	TCS (101)	Software Developer	7.50	50	2025-09-15	2025-09-10
1002	Infosys (102)	System Engineer	6.50	40	2025-09-20	2025-09-15
1003	Deloitte (103)	Business Analyst	8.00	30	2025-09-25	2025-09-20
1004	Amazon (104)	Software Development Engineer	12.00	25	2025-10-05	2025-09-30
1005	Wipro (105)	Project Engineer	6.00	60	2025-10-10	2025-10-05

9.5 Table: eligibility

Column Name	Data Type	Constraints	Description
eligibility_id	BIGINT	PRIMARY KEY	Unique eligibility rule ID
drive_id	BIGINT	FOREIGN KEY, UNIQUE	Drive enforcing rule (1:1 constraint)
minimum_cgpa	NUMERIC(4,2)	NOT NULL	Minimum cutoff CGPA required to apply
maximum_backlogs	INTEGER	NOT NULL	Maximum allowable active backlogs

Sample Tuples (eligibility)

Eligibility ID	Drive ID	Minimum CGPA	Max Backlogs Allowed
201	1001	7.00	0
202	1002	6.50	1
203	1003	7.50	0
204	1004	8.00	0
205	1005	6.00	2

9.6 Table: eligibility_allowed_branch

Column Name	Data Type	Constraints	Description
eligibility_id	BIGINT	COMPOSITE PK, FOREIGN KEY	References `eligibility(eligibility_id)` `ON DELETE CASCADE`
allowed_branch	VARCHAR(30)	COMPOSITE PK	Engineering branch permitted to apply

Sample Tuples (eligibility_allowed_branch)

Eligibility ID	Allowed Branch	Eligibility ID	Allowed Branch
201	CSE	203	IT
201	IT	203	EEE
202	CSE	204	CSE
202	IT	204	IT
202	ECE	205	CSE, IT, ECE, EEE

9.7 Table: hr

Column Name	Data Type	Constraints	Description
hr_id	BIGINT	**PRIMARY KEY**	Unique HR representative identification
company_id	BIGINT	**FOREIGN KEY** (company)	Recruiting enterprise hiring partner
hr_name	VARCHAR(100)	NOT NULL	Full name of HR talent manager
email, phone	VARCHAR(100/15)	NOT NULL / —	HR contact email and phone number

Sample Tuples (hr)

HR ID	Company	HR Name	Email	Phone
301	TCS (101)	Rahul Sharma	rahul.hr@tcs.com	9876500001
302	Infosys (102)	Priya Reddy	priya.hr@infosys.com	9876500002
303	Deloitte (103)	Arjun Kumar	arjun.hr@deloitte.com	9876500003
304	Amazon (104)	Sneha Rao	sneha.hr@amazon.com	9876500004
305	Wipro (105)	Kiran Patel	kiran.hr@wipro.com	9876500005

9.8 Table: applies (M:N Associative Entity)

Column Name	Data Type	Constraints	Description
student_id	BIGINT	**COMPOSITE PK**, **FOREIGN KEY**	Student applying (references `student`)
drive_id	BIGINT	**COMPOSITE PK**, **FOREIGN KEY**	Drive applied to (references `placement_drive`)

Sample Tuples: `(1, 1001)`, `(2, 1001)`, `(3, 1002)`, `(4, 1003)`, `(5, 1004)`.

9.9 Table: application

Column Name	Data Type	Constraints	Description
application_id	BIGINT	**PRIMARY KEY**	Unique application submission serial number
student_id	BIGINT	**FOREIGN KEY** (student)	Applying candidate
drive_id	BIGINT	**FOREIGN KEY** (placement_drive)	Target recruitment campaign
officer_id	BIGINT	**FOREIGN KEY** (placement_officer)	Assigned institutional placement officer
application_date	DATE	NOT NULL	Date on which application was logged
application_status	VARCHAR(30)	NOT NULL	Current lifecycle progression state

Sample Tuples (application)

App ID	Student ID	Drive ID	Officer ID	Date	Status
5001	1	1001	801	2025-09-01	Applied
5002	2	1001	801	2025-09-02	Eligible

App ID	Student ID	Drive ID	Officer ID	Date	Status
5003	3	1002	802	2025-09-03	Applied
5004	4	1003	803	2025-09-04	Shortlisted
5005	5	1004	803	2025-09-05	Interview Scheduled

9.10 Table: interview

Column Name	Data Type	Constraints	Description
interview_id	BIGINT	PRIMARY KEY	Unique interview evaluation ID
application_id	BIGINT	FOREIGN KEY (application)	Associated candidate application
interview_round	VARCHAR(50)	NOT NULL	Stage: 'Technical', 'Managerial', 'HR'
interview_date	DATE	NOT NULL	Date of interview evaluation
result	VARCHAR(30)	NOT NULL	Evaluation outcome: 'Selected', 'Rejected'
remarks	VARCHAR(200)	—	Evaluator panel notes

Sample Tuples (interview)

Interview ID	App ID	Round	Date	Result	Remarks
6001	5004	Technical	2025-09-10	Selected	Good technical knowledge
6002	5005	Technical	2025-09-12	Selected	Strong coding skills
6003	5005	Managerial	2025-09-15	Selected	Good communication
6004	5005	HR	2025-09-18	Selected	Good overall performance
6005	5003	Technical	2025-09-14	Rejected	Technical skills need improvement

9.11 Table: offer

Column Name	Data Type	Constraints	Description
offer_id	BIGINT	PRIMARY KEY	Unique placement job offer letter ID
student_id	BIGINT	FOREIGN KEY (student)	Candidate receiving job offer
company_id	BIGINT	FOREIGN KEY (company)	Offering corporate organization
offer_date	DATE	NOT NULL	Date offer was formally released
package_lpa	NUMERIC(6,2)	NOT NULL	Confirmed annual remuneration in LPA
offer_status	VARCHAR(30)	NOT NULL	Decision: 'Accepted', 'Pending', 'Rejected'

Sample Tuples (offer)

Offer ID	Student ID	Company	Offer Date	Package (LPA)	Status
7001	4	Deloitte (103)	2025-09-20	8.00	Accepted
7002	5	Amazon (104)	2025-09-25	12.00	Pending

10. CONCLUSION AND FUTURE SCOPE

10.1 Project Conclusion

The **PlaceX College Placement Management Portal** successfully resolves the inefficiency, opacity, and vulnerability of traditional spreadsheet-driven campus placement administration. Grounded in rigorous relational database theory, the project achieved several milestones:

- **Robust Relational Architecture:** Formulated an 11-relation schema seeded with over 12,000 realistic student academic records, establishing ACID-compliant data persistence on PostgreSQL.
- **Proven BCNF Compliance:** Decomposed non-atomic multi-valued attributes into normalized structures, eliminating partial and transitive dependencies while ensuring referential integrity via cascade delete configurations.
- **Automated Eligibility Validation:** Implemented dynamic SQL and backend validation routines that evaluate student CGPA, backlogs, and academic disciplines against corporate criteria automatically.
- **High-Yield Analytical Querying:** Constructed an optimized query suite computing complex aggregations, branch-wise salary metrics, multi-round interview progressions, and placement KPI summaries.
- **Working Full-Stack System with Completed Search & Filtering:** Connected the PostgreSQL database engine to an asynchronous Node.js/Express REST API and responsive frontend, incorporating a fully completed real-time multi-attribute search and filtering engine.

10.2 Future Scope & Planned Enhancements

While the current PlaceX implementation delivers a production-ready core DBMS foundation, future iterations can expand the portal with the following enhancements:

1. **Database Trigger-Based Hard Restraints:** Implement PostgreSQL trigger functions (`BEFORE INSERT ON application`) to automatically block application attempts at the database engine level if candidate credentials do not satisfy drive cutoffs.
2. **AI-Driven Resume Parsing & Skill Extraction:** Integrate Natural Language Processing (NLP) models to extract competencies from uploaded PDF resumes and automatically compute alignment scores against corporate job descriptions.
3. **Automated Notification & Webhook Pipeline:** Implement email and SMS notifications (via Twilio and Nodemailer) to alert students regarding upcoming drive deadlines, interview scheduling, and offer letter issuances.
4. **Enterprise JWT Authentication & Cryptographic Hashing:** Upgrade credential management to salted bcrypt password hashing and signed JSON Web Tokens (JWT) for secure role-based session authorization.

10.3 Visual Appendix: Working Portal Demonstration

The figures below illustrate the operational PlaceX full-stack application connecting the PostgreSQL database, Express API, and modern responsive user interface:

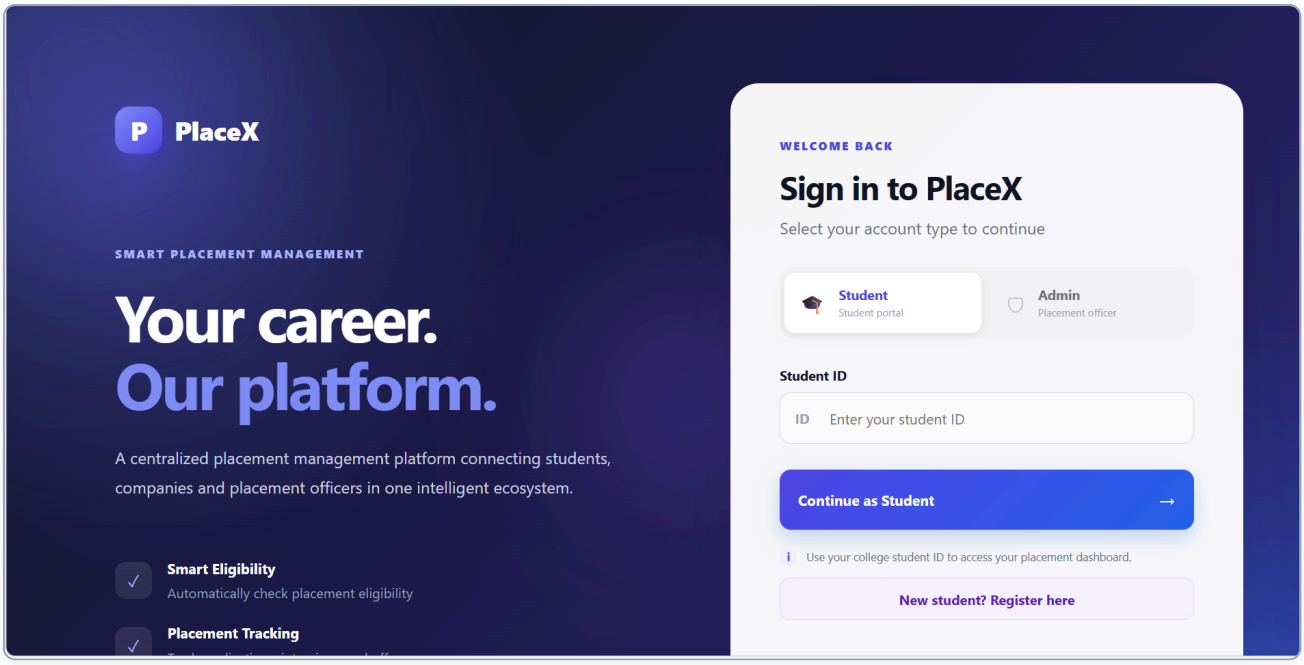


Figure 10.1: PlaceX Smart Portal Authentication Interface supporting Student and Placement Administrator access.

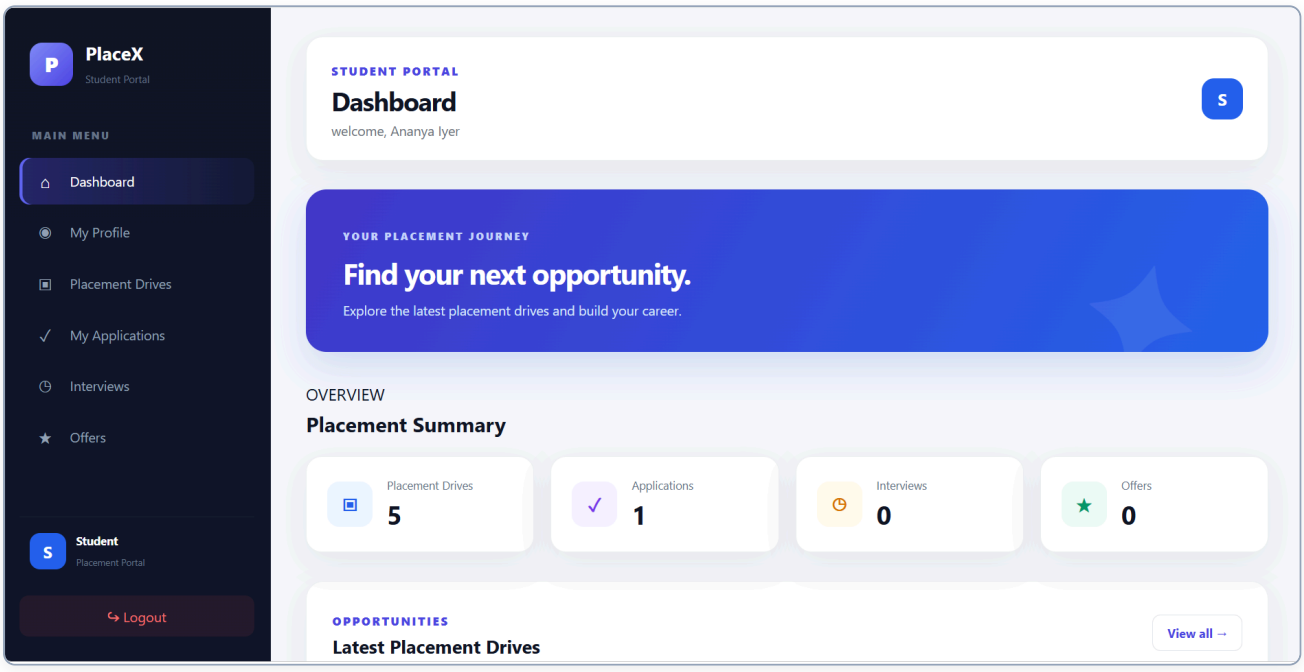


Figure 10.2: PlaceX Student Command Center displaying active placement drives, application status, and interview rounds.

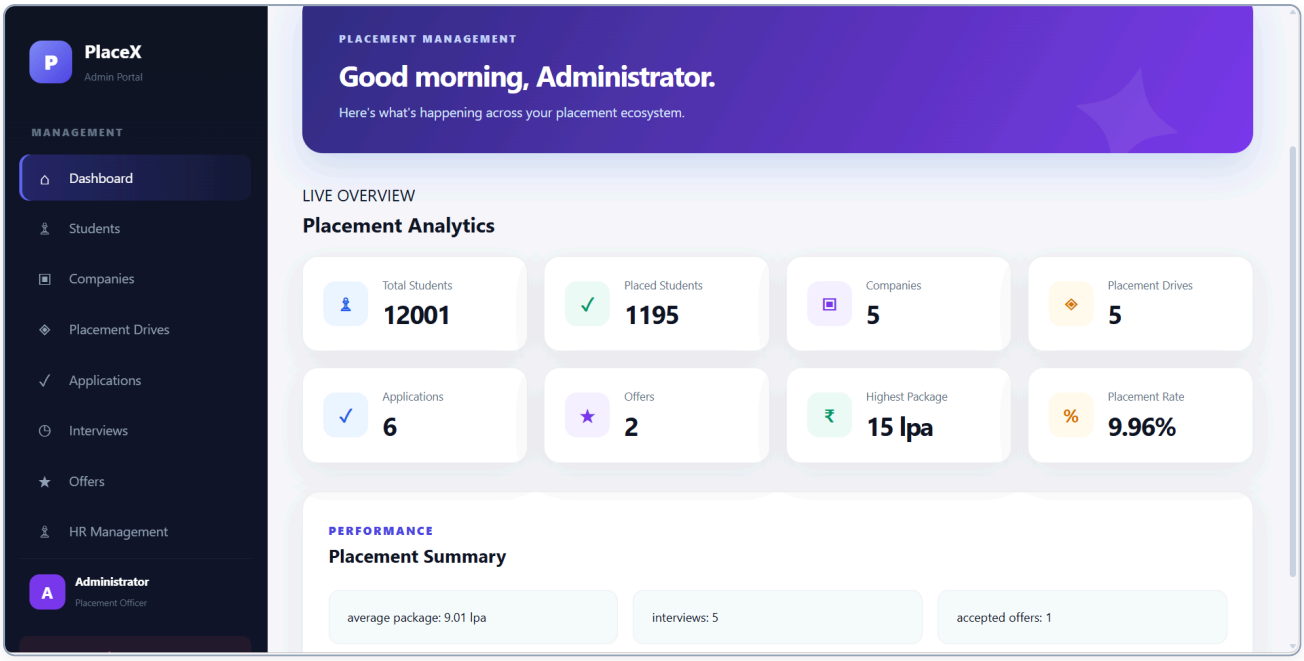


Figure 10.3: Administrator Live Analytics Dashboard rendering real-time placement KPIs, CTC averages, and student/drive management tables.